

作业感受

1. 我如何完成第一次挑战

我先根据挑战说明拆分任务：准备 SGLang 推理环境，启动 OpenAI-compatible 服务，确认 `/v1/models` 可以访问并完成一次推理；然后阅读 SGLang 的 RadixAttention 论文和 vLLM 的 PagedAttention 论文，整理 KV Cache、Prefix Reuse、Paged KV 等概念；最后读取 Mooncake trace 前 10 条记录，保留原始长度字段，将统一截取为 3000 个客户端 token 的合成输入发送到服务中，输出上限限制在 8-32 token，并记录实际输入输出 token 数、状态和延迟。

核验原始数据后，重试轮 10 条请求均成功，服务报告输入均为 3008 token，成功请求平均 latency 为 0.6294 秒。客户端 3000 与服务端 3008 的差值尚不能确定来源。本轮没有保持 trace 原始长度分布；请求完成后才进行指数随机等待，因此不是独立 Poisson 到达过程，不能宣称已满足挑战的全部实验要求。随后修正脚本并完成独立 Poisson 重跑，结果见下文。

完成过程中，我把“运行验证”和“概念理解”交替进行。先用最小请求确认环境和服务链路可用，再通过论文、官方文档和运行结果理解每个组件在请求流程中的位置。对容易混淆的概念，我用我的话重新画出“请求接收 → 调度 → Prefill → KV Cache/RadixCache → Decode → 采样与流式输出”的流程，并将其整理到理论回答中。

2. 最困难的部分与解决方法

最困难的是环境配置和专业名词理解。环境方面，SGLang、Ray、模型版本、GPU/CUDA 以及 Python 依赖之间存在匹配关系，任何一项不一致都可能导致服务无法启动。我的处理方式是先固定版本和模型，再逐步检查依赖、服务进程和 HTTP 接口；每完成一层就用 `/v1/models` 或一次最小推理请求验证，而不是一次性排查所有问题。

概念方面，RadixAttention、RadixCache、page-sized KV cache、PagedAttention 都涉及 KV Cache，但关注层次不同，初看时容易混为一谈。我通过对照论文中的系统流程、实现术语和 vLLM 文档，把问题拆成两层：一层是“哪些前缀可以复用”，另一层是“KV 在显存中如何分页存放和读取”。这样可以明确 RadixAttention 主要负责前缀匹配与复用策略，Paged KV/PagedAttention 主要负责 KV 的物理组织和访问。

3. 对“从 0 到 1”科研工作的认识与启发

这次任务让我认识到，从 0 到 1 的科研工作不只是阅读论文或运行代码，而是要把问题定义、环境搭建、实验设计、结果记录和概念解释连成一个可复现的闭环。论文提供方法和背景，代码实验检验方法是否能运行，日志和截图提供证据，最后还需要用清楚的语言说明结果意味着什么。

对今后科研工作的启发有三点。第一，先把大问题拆成可以验证的小问题，每一步都留下可检查的中间结果。第二，遇到专业术语时不能只记定义，要追问它在系统流程中解决了什么问题、输入输出是什么、和相近概念边界在哪里。第三，实验记录和失败过程同样重要，它们能帮助定位问题，也能让最终结论具有可复现性。AI 可以加快检索和解释，但研究者仍需自己确认来源、设计验证方法并对结论负责。

修正实验与核验

修正实验已实际完成：从输入 1-3000 token、输出 1-512 token 的 2708 条候选 trace 中，用随机种子 20261008 采样 10 条，保留各条原始输入长度与输出上限。独立累计指数间隔生成 Poisson 到达时间， $\lambda=0.5$ 请求/秒；客户端按计划发送，不等待上一请求完成。10/10 请求成功，平均 latency 为 1.3982 秒，最大调度偏差为 4.38 毫秒。实际输出允许因 EOS 提前结束，已记录 finish_reason。服务输入计数与本地 chat template 一致，均比合成正文多 8 token；此核验仅适用于新轮次。没有测量 TTFT 或缓存性能，不能据此宣称论文优化收益。

首次重跑因默认 FlashInfer 采样后端缺少 nvcc 而失败；改用 PyTorch 采样后端后重试成功。失败轮独立保留，不混入成功统计。单次接口推理返回 HTTP 200，只证明调用链路可用，不证明回答内容正确。